

Курсовая работа

**Современные средства
и методы Web-разработки**

студенты 3-го курса ФИТ-231

**Сергиенко Александр Сергеевич
Кононенко Егор Сергеевич**

CarGO

Сервис аренды автомобилей



Главная цель проекта →

Проектирование и разработка современного, масштабируемого веб-приложения для автоматизации бизнес-процессов сервиса проката автопарка с жесткими правилами транзакционного контроля бронирования

Практическая значимость →

Исключение человеческого фактора при распределении машин.

Прозрачный расчет стоимости по гибким тарифам.

Оперативный контроль технического состояния автопарка.

Защита от double-booking (двойного бронирования).

01

Проектирование архитектуры и СУБД

Разработка ER-схемы данных в 3NF, настройка ограничений целостности.

02

Реализация надежного REST API бэкенда

Бизнес-логика на Kotlin + Spring Boot с защитой от race condition.

03

Разработка клиентского SPA интерфейса

Удобное реактивное веб-приложение на Vue 3 + Pinia + TypeScript

Гибкий алгоритм динамического расчета цены (Dynamic Pricing)

Итоговая цена аренды = Базовая ставка за бронирование + Посуточная ставка тарифа * (Количество суток аренды).

Тарифы жестко привязаны к классам машин. При оформлении заказа рассчитывается полная сумма с учетом возраста/стажа.



Учет активов (Car)

VIN-код автомобиля

Гос. регистрационный знак

Класс комфорта (Эконом, Бизнес, Люкс)

Статус доступности (активен, ТО, выведен)



Конфликты дат (Overlaps)

Запрет двойного бронирования

Пересечение интервалов дат
проверяется на сервере в рамках
транзакции базы данных

Входящий запрос с наложением
отклоняется

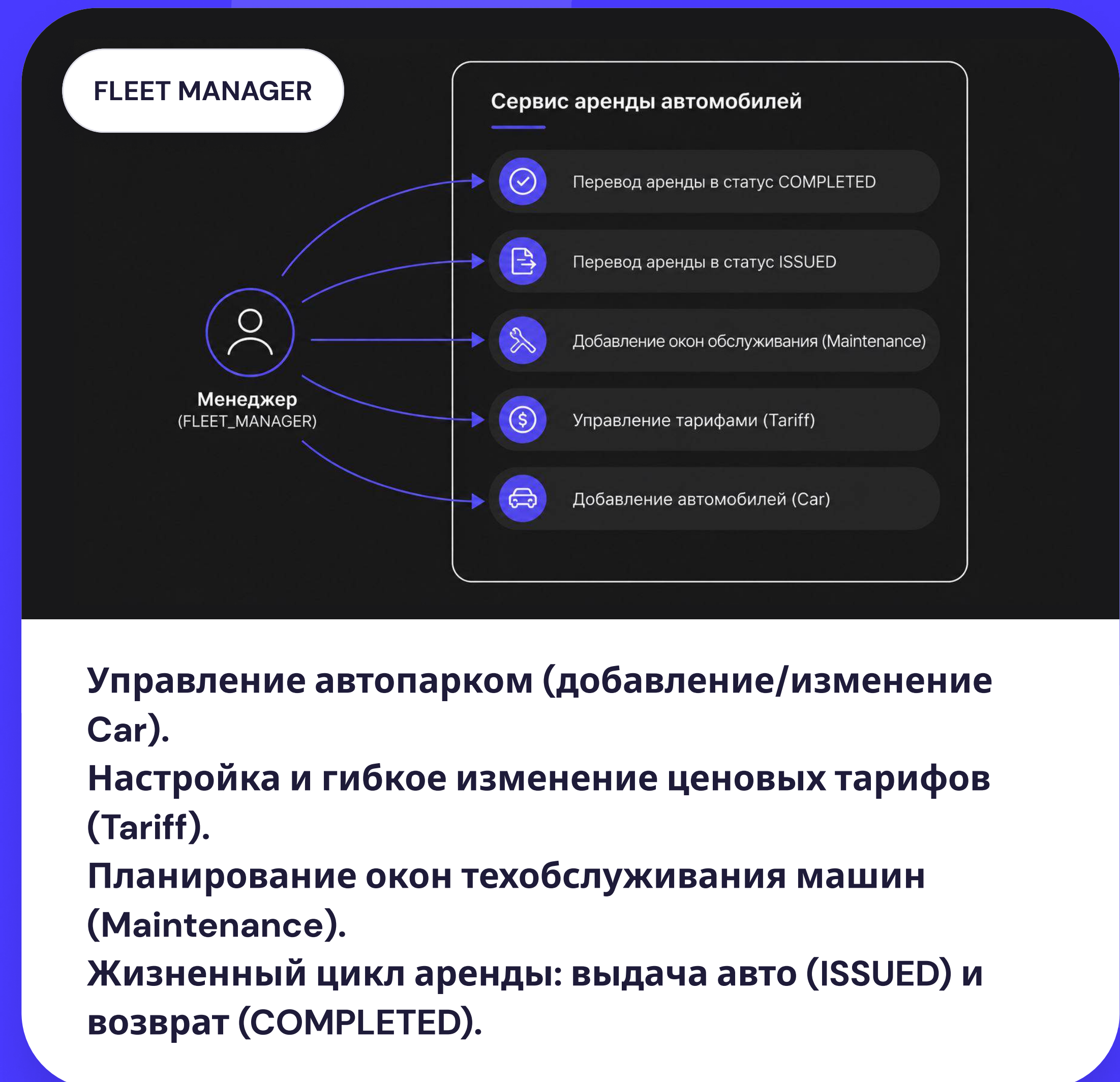
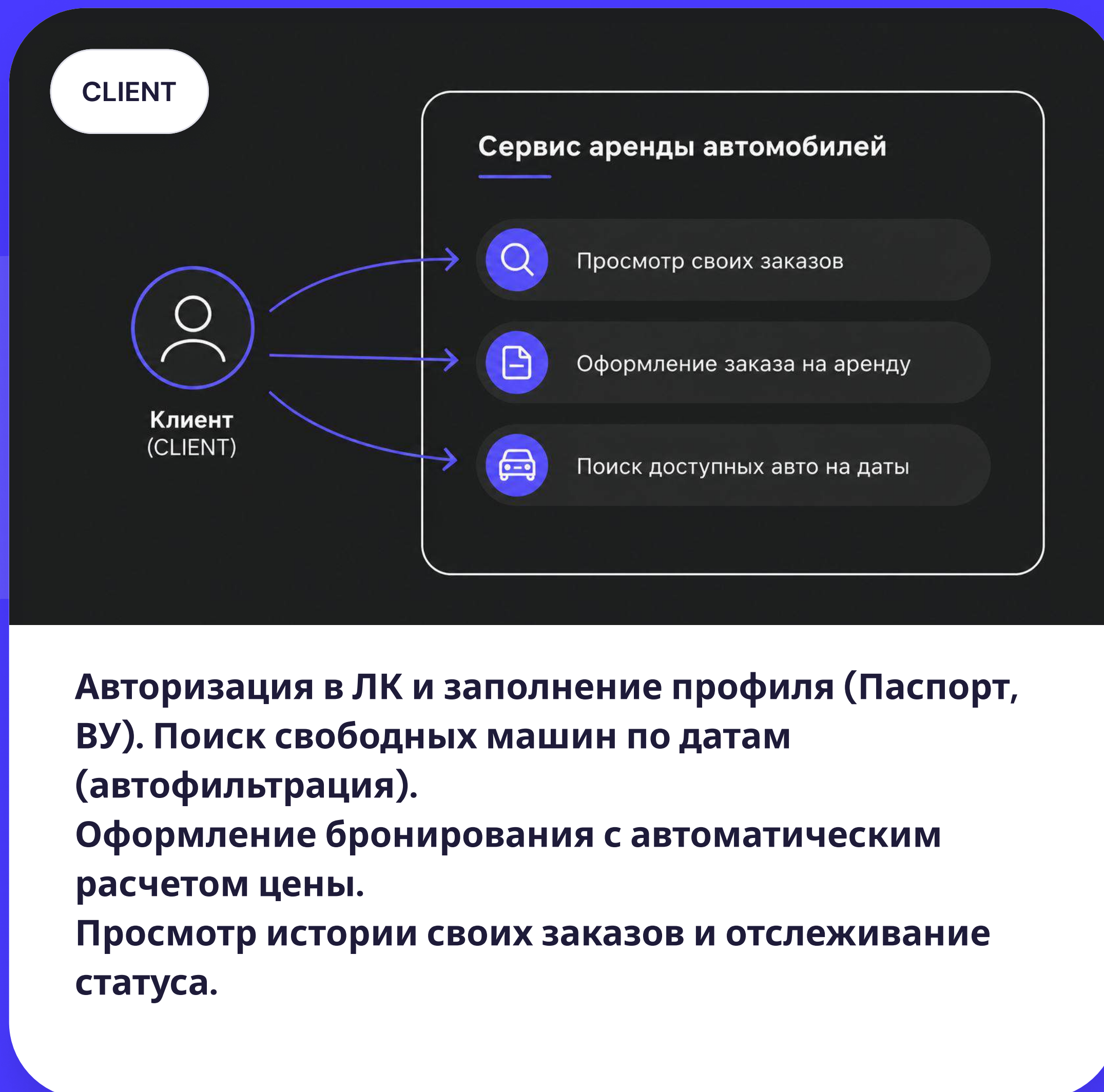


Окна обслуживания (Maintenance)

Планирование ТО менеджером

Запись блокирует авто на период
работ

Окно обслуживания равносильно
активной аренде при проверке
занятости



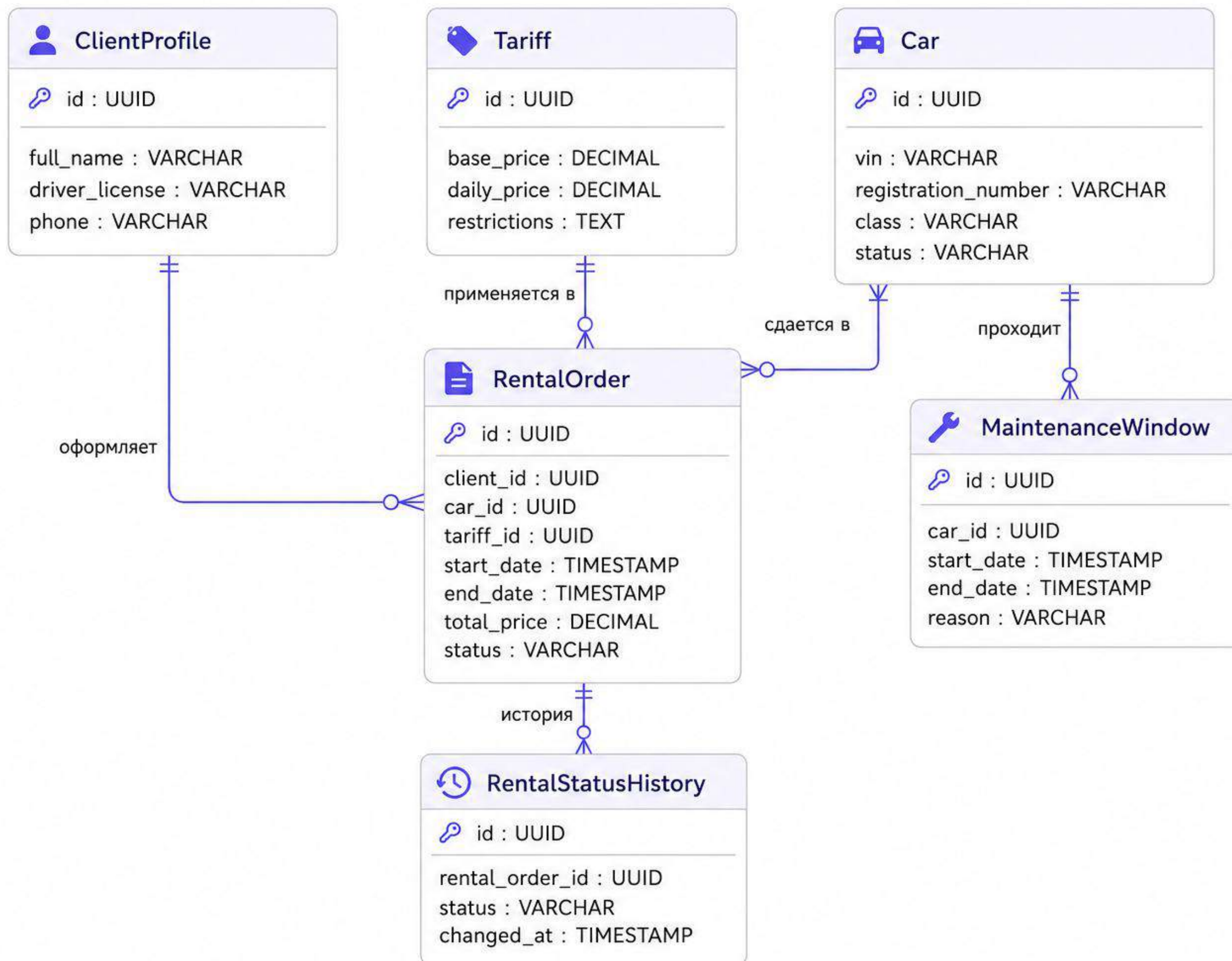


Схема данных в 3NF

База данных полностью нормализована до 3NF для устранения аномалий обновления.

Ключевые таблицы:

Car: Данные авто и VIN-код

Tariff: Расчет стоимости аренды

ClientProfile: Профиль и ВУ

RentalOrder: Связующий заказ

MaintenanceWindow: Окна ТО

RentalStatusHistory: История статусов

Ссылочная целостность:

Обеспечена каскадными операциями и внешними ключами в PostgreSQL.

Meet our amazing team



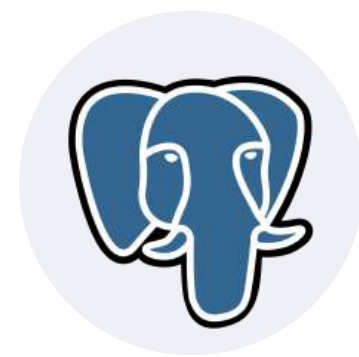
Kotlin

Серверный язык
с компиляторным null
safety и лак-чным кодом



Spring Boot

ORM (Hibernate/JPA),
Spring Security, валидация
DTO, REST API



PostgreSQL

Реляционная СУБД с
жесткими блокировками
при бронировании дат



Vue 3

Реактивный SPA-клиент.
Быстрый рендеринг,
компонентный подход



TypeScript / Vite

Статическая типизация на
фронте и мгновенная
сборка статики



Docker

Полная контейнеризация
всех компонентов
для мгновенного запуска

Проблема: Double-Booking

В высоконагруженных системах два пользователя могут **ОДНОВРЕМЕННО** нажать кнопку «Забронировать» на один автомобиль на одни и те же даты.

Классическая проверка «сначала проверь в коде, потом вставь» приводит к состоянию гонки (Race Condition).

Последствия:

Один автомобиль выдается двум разным клиентам, финансовые и репутационные потери.

Двухэтапная атомарная проверка с блокировкой

- 01 Декларативная транзакция Spring (@Transactional):**
Изоляция транзакций обеспечивает целостность проверки в рамках одного подключения к БД.
- 02 Высокопроизводительный запрос проверки наложений:**
Запрос проверяет пересечения запрашиваемого интервала с существующими заказами:
`(start_date <= :requested_end) AND (end_date >= :requested_start)`
И одновременно проверяет пересечения с окнами обслуживания:
`(m_start <= :requested_end) AND (m_end >= :requested_start)`
- 03 Пессимистическая блокировка (Pessimistic Write Lock):**
Используется блокировка строки выбранного автомобиля в СУБД PostgreSQL. Второй конкурентный поток переходит в ожидание до завершения первой транзакции. Race Condition полностью устранен!
- 04 Расчет цены:**
Выполняется строго после валидации дат, гарантируя корректное применение тарифа.

Аутентификация & Авторизация

Использование Spring Security.
Безсессионный доступ Stateless REST.

Генерация криптографически
подписанных JWT-токенов доступа.

Ролевое разграничение (CLIENT /
FLEET_MANAGER) на уровне
эндпоинтов API.

Безопасность хранения данных

Одностороннее хэширование паролей
пользователей с использованием соли
перед сохранением в PostgreSQL.

Многоуровневая валидация входящих
параметров (DTO Validation).

Защита от SQL-инъекций за счет
использования Hibernate и
параметризованных запросов.

Аудит & Логирование жизненного цикла

Любой переход заказа между
статусами фиксируется в отдельной
таблице RentalStatusHistory.

Логировается точное время изменения
статуса и ID инициатора.

Полная прозрачность аудита при
возникновении споров с клиентами.

Итоги и результаты работы

Основные результаты проекта

1. Спроектирована и нормализована до 3NF база данных сервиса аренды авто.
2. Реализована бизнес-логика защиты от двойного бронирования и наложений на интервалы технического осмотра автотранспорта.
3. Разработано адаптивное веб-приложение (SPA клиент + REST бэкенд).
4. Обеспечена безопасность данных (JWT, Spring Security, хэширование паролей).
5. Проведена полная контейнеризация через Docker и Docker Compose для легкого деплоя

Направления для развития

1. Интеграция платежных шлюзов:
Автоматическое проведение транзакций оплаты проката при бронировании с заморозкой залога (холдирование средств).
2. Модуль автоуведомлений клиентов:
Интеграция Telegram(МАКС)-ботов или SMS-шлюзов для напоминания о скором начале или завершении аренды.
3. Мобильное приложение водителя:
Перенос клиентского опыта в мобильное приложение для удобного поиска машин на карте.

